# Beyond Chunk-Then-Embed: A Comprehensive Taxonomy and Evaluation of Document Segmentation Strategies for Information Retrieval

No Author Given

No Institute Given

**Abstract.** Document chunking is a critical preprocessing step in dense retrieval systems, yet the design space of chunking strategies remains poorly understood. Recent work has proposed novel approaches ranging from LLM-guided chunking methods such as DenseX [**?**] and Lumber-Chunker [**?**], to contextualized chunking strategies like Late Chunking [**?**], which generates embeddings before segmentation to preserve contextual information. However, existing methods have been evaluated in isolation across different benchmarks, making direct comparisons difficult.
This paper presents a systematic framework that unifies existing chunking strategies along two key dimensions: (1) the *order* of chunking and embedding operations (pre-embedding vs. contextualized chunking), and (2) the *segmentation methods* (traditional fixed-size, sentence/paragraph-based, semantic boundary-based, and LLM-guided approaches). We reproduce and evaluate representative methods from each category across two distinct retrieval paradigms: single-document needle-in-a-haystack retrieval and multi-document web-scale retrieval. Our comprehensive evaluation reveals that ... Our code and evaluation benchmarks are publicly available at `https://xx`.

**Keywords:** Chunking · Document Ranking · Re-ranking.

## 1 Introduction

Dense retrieval systems, which represent queries and documents as low-dimensional vectors, have become a core part of modern information retrieval. They power a wide range of applications, from web search engines to advanced retrieval-augmented generation systems. A key challenge in using these systems is fitting long documents into the fixed-size input windows of embedding models. This requires **document chunking**: the process of dividing documents into smaller, meaningful units for indexing and retrieval. The choice of a chunking strategy is not just a preprocessing step; it is an important decision that greatly affects how well retrieval works. Chunks that are too small may not have enough context to answer a query, while oversized chunks can make it harder to find the relevant information and may be too large for the model to process.

Chunking has developed in two main ways: the method of segmentation and the approach to embedding. Traditionally, systems use a *pre-embedding chunking* approach, where documents are first divided and each resulting chunk is then encoded on its own. Within this approach, segmentation methods range from simple fixed-size windows to more advanced techniques based on linguistic boundaries (e.g., sentences or paragraphs [**?**]), changes in meaning [**?**], and even guidance from large language models (LLMs) [**?**,**?**]. More recently, the introduction of embedding models with longer context windows that support thousands of tokens [**?**,**?**,**?**] has allowed for a new *contextualized chunking* approach. Methods like Late Chunking [**?**] are a good example of this change: they first encode a whole document to create context-aware embeddings and only then divide it, keeping useful information that connects the chunks.

Despite this rapid development, progress has been disorganized because there has been no standard way to evaluate these methods. New chunking strategies are often tested in different and unconnected ways, making it impossible to compare them directly. For example, proposition-based methods were evaluated on question-answering datasets using models with a 512-token limit [**?**]. In contrast, LLM-guided methods like LumberChunker were benchmarked on a single-book retrieval task using private embedding APIs [**?**], while Late Chunking was tested on standard BEIR passage retrieval benchmarks [**?**]. These differences cover embedding models, datasets, and even the type of retrieval task (e.g., searching through many documents vs. a single document). As a result, it is unclear whether the differences in performance come from the actual strengths of the chunking strategies themselves or from these other factors in the experiments.

To address these gaps and provide a clear, evidence-based understanding of document chunking, we conduct a careful study guided by the following research questions:

**RQ1** *How do different segmentation methods compare in standard multi-document retrieval?* We evaluate fixed-size, sentence/paragraph-based, semantic boundary-based, and LLM-guided chunking methods using traditional pre-embedding chunking on standard BEIR benchmarks.

**RQ2** *Do segmentation methods generalize across retrieval paradigms?* We apply the same segmentation methods to single-document needle-in-a-haystack retrieval, testing whether the relative strengths observed in multi-document settings hold when the retrieval scope is constrained to individual documents.

**RQ3** *Can contextualized chunking provide consistent improvements across different configurations?* We apply contextualized chunking to all segmentation methods from RQ1 across both retrieval paradigms from RQ2 to determine whether contextualized chunking provides universal benefits.

**RQ4** *What factors determine when contextualized chunking outperforms pre-embedding approaches?* We implement Late Chunking as a representative contextualized method and systematically vary chunk size, overlap ratios, document types, and query characteristics to measure both retrieval effectiveness and computational overhead.

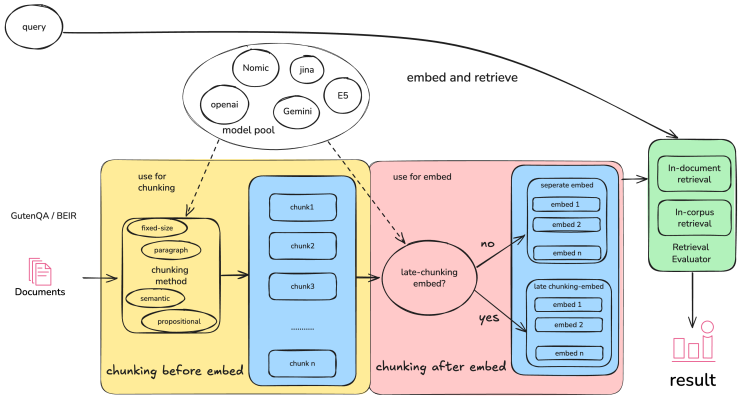


Fig. 1: Unified framework for evaluating document chunking strategies across retrieval paradigms. Documents from BEIR and GutenQA datasets are segmented using one of several methods. Depending on the configuration, embeddings are generated either before chunking (*pre-embedding chunking*) or after full-document encoding (*contextualized chunking*, e.g., Late Chunking). Resulting chunk embeddings are indexed and evaluated under two retrieval settings: (1) *in-document retrieval* (needle-in-a-haystack) and (2) *in-corpus retrieval* (multi-document). The pipeline supports multiple embedding and generative model families, enabling controlled comparison across chunking paradigms and model capacities.

Through investigating these research questions, we (1) establish comprehensive baselines for segmentation methods in multi-document retrieval (**RQ1**), (2) identify which chunking strategies are robust across different retrieval paradigms (**RQ2**), (3) assess whether contextualized chunking provides universal improvements (**RQ3**), (4) uncover the key factors that determine when contextualized chunking justifies its computational overhead (**RQ4**), and (5) release our complete implementation and evaluation framework to facilitate reproducibility and future research.

## 2 Benchmarking Document Chunking Strategies

Document chunking is the critical step that transforms raw text into the retrievable units used by dense retrieval systems. To systematically compare the wide range of available methods, we organize them along two key dimensions: (1) **segmentation strategy**, which defines how textual boundaries are identified, and (2) **embedding paradigm**, which determines whether chunking occurs before or after the text is encoded into vector representations. This framework provides a clear structure for our experiments and clarifies how different techniques relate to one another.

### 2.1 Taxonomy of Segmentation Strategies

We classify segmentation strategies into two main families: structure-based heuristics that rely on the document's explicit layout, and semantically-informed methods that use language understanding to find logical divisions.

**Structure-Based Heuristics** These methods use observable textual features to create chunks. They are widely adopted in practice due to their speed, simplicity, and deterministic output. We evaluate three common heuristic strategies:

- **Paragraphs:** This strategy splits documents at newline characters and removes empty segments. It respects the document's intended structure at no extra processing cost and is often effective for well-organized texts like news articles.
- **Fixed-Size Windows:** This method divides text into segments of a constant token length (e.g., 512 tokens), with no overlap between consecutive chunks. While computationally efficient, it can unnaturally split coherent thoughts or sentences, potentially harming the quality of the resulting embeddings [**?**].
- **Sentences:** This approach uses a regex-based splitter to identify sentence boundaries and groups multiple sentences per chunk. This creates moderately-sized segments that balance granularity with context preservation, suitable for tasks where answers may span multiple sentences.

**Semantically-Informed and LLM-Guided Strategies** These advanced methods use models to analyze the text's meaning and identify more natural breakpoints. The aim is to produce chunks that are internally coherent, providing a better foundation for retrieval.

- **Semantic Sentence Splitting:** This technique calculates semantic similarity between adjacent sentences using embeddings. When the similarity drops below a threshold, indicating a topic shift, a boundary is inserted [**?**]. This adapts to the flow of the document's content.
- **Proposition-Based Chunking:** This method uses an LLM to break down text into atomic facts or "propositions" [**?**]. Each proposition becomes a chunk, which creates highly specific units ideal for factual lookups. However, this can significantly increase the total number of chunks and is computationally expensive due to its reliance on LLM inference.
- **LumberChunker:** This approach directly instructs an LLM to segment the document by inserting breakpoints between paragraphs where it detects a topic shift [**?**]. While powerful, this method is also slower and more expensive due to its use of LLM inference.

It is important to note that semantically-informed and LLM-guided strategies often build upon structure-based heuristics. For example, semantic sentence splitting relies on an initial sentence segmentation, while both proposition-based chunking and LumberChunker typically operate on paragraph-level text.

### 2.2 Pre-Embedding vs. Contextualized Chunking

The second dimension of our framework is **Embedding Paradigms**: the order in which chunking and embedding are performed. The *pre-embedding* paradigm, which has been the traditional approach, segments the document first and then encodes each chunk in isolation. This design is simple and scalable but loses all contextual information across chunks.

In contrast, the *contextualized chunking* paradigm (also known as late chunking) reverses this order. First, a long-context encoder processes the entire document, generating token-level embeddings that are aware of the global context [**?**]. Only then are chunk boundaries applied. The final embedding for each chunk is produced by pooling (e.g., averaging) the embeddings of the tokens it contains. This approach preserves cross-chunk relationships at the cost of higher computational and memory requirements.

### 2.3 Evaluation Tasks

As noted in the introduction, prior work has used inconsistent settings, making direct comparisons difficult. To ensure a fair evaluation, we test each chunking strategy across the two dominant retrieval paradigms:

- **In-Document Retrieval:** Also known as"needle-in-a-haystack", this task involves finding a specific piece of information within a single long document. It tests a method's ability to precisely locate content.
- **In-Corpus Retrieval:** This is the classic multi-document retrieval task, where the goal is to find the most relevant documents from a large collection, such as the BEIR benchmarks [**?**]. This tests for scalability and effectiveness in a broader search environment.

By systematically evaluating each chunking method across both paradigms with a consistent set of embedding models and controls, our study isolates the impact of the chunking strategy itself, providing a clearer understanding of its true effect on retrieval performance.

## 3 Experimental Setup

We evaluate each chunking strategy using the unified framework described in Section 2. This section details the specific implementations, datasets, and evaluation procedures used in our experiments.

### 3.1 Segmentation Methods

To ensure a fair comparison, we use a consistent implementation for each segmentation strategy across all datasets.

For **structure-based heuristics**: the **Paragraphs** strategy splits documents at newline characters and removes empty segments. The **Fixed-Size**

method divides documents into uniform 256-token segments with no overlap. The **Sentences** approach uses a regex-based splitter and groups five sentences per chunk by default, following the implementation from LateChunking [**?**].

For **semantically-informed and LLM-guided strategies**, the **Semantic Sentence** strategy employs llama-index's `SemanticSplitterNodeParser` with `jina-embeddings-v2-small-en`, also following the LateChunking work [**?**], to compute cosine similarity between adjacent sentences and insert boundaries based on similarity thresholds. For the **Proposition-Based** strategy, we utilize `Gemini-2.5-Flash` for processing (implementation details in supplementary materials). The **LumberChunker** strategy uses a configurable generator system with `Gemini-2.5-Flash` for fair comparison across LLM-based methods. For all LLM-based methods, we set the generation temperature to 0.0 to ensure deterministic output.

### 3.2 Embedding Models Used

To evaluate the effectiveness of different segmentation strategies and embedding paradigms, we select the following four models:

- **jina-embeddings-v2-small-en(Jina-v2)** [**?**]: A lightweight model optimized for efficiency while maintaining competitive performance.
- **jina-embeddings-v3(Jina-v3)** [**?**]: An updated version with improved multilingual capabilities and semantic understanding.
- **nomic-embed-text-v1(Normic-v1)** [**?**]: A model designed for high-quality dense retrieval with strong performance on diverse benchmarks.
- **multilingual-e5-base(E5-base)** [**?**]: A multilingual model using mean pooling for text representation.

While the first three models were evaluated in the original LateChunking work [**?**], we additionally include multilingual-e5-base to test the robustness of our approach. Since LateChunking requires models that use average pooling for representation, our selection was constrained to the highest-performing embedding models on the MTEB leaderboard that employ mean pooling for text representation.

All embedding models handle the *pre-embedding* paradigm, where documents are first chunked and then each chunk is encoded separately. For *contextualized chunking*, which requires encoding multiple chunks together, we employ a maximum context utilization strategy: we feed the maximum possible number of chunks that fit within the embedding model's context window. When the document exceeds this limit, we create a new context window for the remaining chunks.

### 3.3 Datasets and Evaluation

Our evaluation is designed to test the generalizability of chunking strategies across different retrieval tasks and domains.

**In-Document Retrieval** We evaluate in-document retrieval using the **GutenQA** dataset [**?**], which contains 100 classic literature books. For each book, a set of questions asks for specific information contained within that book. Overall, there are 3,000 question-answer pairs derived from 100 books. This task tests for precise information localization within a single, long document.

The ground truth in GutenQA is annotated at the paragraph level. Since chunks may contain partial paragraphs or span multiple paragraphs, we consider a retrieved chunk as relevant if it overlaps with any ground-truth paragraph. We evaluate retrieval performance by examining whether the top-n retrieved chunks contain the relevant information.

For evaluation, we report DCG@10 (Discounted Cumulative Gain at rank 10), following exisiting work from LumberChunker [**?**], calculated as:

$$\text{DCG@10} = \sum_{i=1}^{10} \frac{2^{\text{rel}_i} - 1}{\log_2(i+1)}$$

where $\text{rel}_i$ is the relevance score of the result at rank $i$.

**In-Corpus Retrieval** We evaluate in-corpus retrieval on a selection of datasets from the **BEIR** benchmark [**?**]: FiQA, ArguAna, SciDocs, TREC-COVID, SciFact, and NFCorpus. We chose these specific datasets based on two criteria. First, they cover a diverse range of retrieval tasks and domains, from finance and scientific articles to biomedical texts. Second, these datasets have a relatively small number of documents, which was a practical consideration to manage the computational cost associated with the LLM-guided chunking methods.

For in-corpus retrieval, ground truth is annotated at the document level. Following established practices [**?**], we use the score of the highest-ranked chunk from each document as the document's relevance score. This max-pooling approach has been shown to be the most effective strategy for aggregating chunk-level scores to document-level rankings.

For evaluation, we report nDCG@10 (normalized Discounted Cumulative Gain at rank 10) for all BEIR tasks. The nDCG@10 metric normalizes the DCG@10 score by the ideal DCG@10 score, producing a value between 0 and 1.

## 4 Result

### 4.1 RQ1: Impact of Segmentation Strategies in In-Corpus Retrieval

To establish baseline effectiveness characteristics of different segmentation strategies in realistic in-corpus retrieval scenarios, we evaluated six segmentation methods across six BEIR datasets using four embedding models under the pre-embedding chunking paradigm. This research question addresses the core practical concern: which segmentation approach performs best when retrieval occurs across a corpus of documents?

**No single chunking strategy dominates across all embedding models and datasets.** Across the six in-corpus retrieval tasks, paragraph-based chunking achieves the highest average nDCG@10 for three of the four embedding models (Jina-v2: 0.4303, Jina-v3: 0.4948, E5-base: 0.4997), while fixed-size chunking performs best with Nomic embeddings (0.4828). This variability suggests that the optimal segmentation strategy is contingent on how different embedding models encode semantic boundaries and the inherent structure of the target domain.

**Semantically-informed and LLM-guided strategies underperform relative to their computational cost.** Contrary to expectations that more sophisticated semantic analysis would improve retrieval effectiveness, advanced methods consistently underperform simpler heuristics. Proposition-based chunking yields the lowest average nDCG@10 across all models (ranging from 0.3487 to 0.3916), representing a 15–20% performance degradation compared to paragraph-based chunking. LumberChunker ranks fourth or fifth in average corpus effectiveness across datasets. For instance, with Jina-v3, paragraph chunking outperforms LumberChunker by 11% (0.4948 vs 0.4463) and proposition-based chunking by 26% (0.4948 vs 0.3916). Given the significantly higher computational cost of LLM-based methods, these results suggest that the added complexity does not translate to retrieval gains in multi-document settings.

**Dataset characteristics and domain-specific structure influence segmentation effectiveness.** The relative performance of segmentation methods varies meaningfully across different corpus types. For domain-specific datasets like SciFact and TREC-COVID, sentence-based chunking often performs competitively or best (e.g., SciFact with Jina-v2: 0.6548; TREC-COVID with Nomic: 0.7859), suggesting that shorter, semantically-bounded units work well when documents have highly specialized terminology and structure. In contrast, for argumentative text (ArguAna), paragraph and fixed-size chunking consistently outperform more sophisticated strategies, indicating that natural discourse boundaries are crucial for certain document types. This pattern suggests that segmentation strategy selection should account for the structural characteristics of the target corpus.

**Embedding model architecture affects sensitivity to segmentation choice.** The variance in effectiveness across segmentation methods differs substantially between embedding models, indicating that not all models are equally robust to suboptimal chunk boundaries. Jina-v2 exhibits the highest sensitivity with a 23% effectiveness gap between best and worst strategies (0.4303 vs 0.3487), while E5-base demonstrates greater robustness with only an 11% gap (0.4997 vs 0.3883). This finding suggests that some embedding models are inherently better equipped to handle suboptimal segmentation, which is an important consideration when deploying retrieval systems.

**Overall, structure-based heuristics provide the most reliable and practical approach for in-corpus retrieval.** Paragraph-based chunking emerges as the most robust general-purpose strategy, achieving the highest average nDCG@10 in three of four models and consistently ranking in the top-2 across individ-

ual datasets. Fixed-size chunking offers comparable effectiveness (within 2% of paragraph chunking on average) with the added computational advantage of deterministic and parallelizable processing. Surprisingly, despite their conceptual sophistication, LLM-guided strategies provide diminishing returns: proposition-level chunking averages 15–20% lower effectiveness than paragraph chunking across all models, while incurring substantially higher computational costs. These results directly address RQ1 by demonstrating that for standard in-corpus retrieval, simpler segmentation methods that either respect natural document structure or maintain consistency provide superior effectiveness and robustness compared to semantic-aware alternatives, and do so with significantly lower computational overhead.

### 4.2 RQ2: Do Segmentation Strategies Generalize Across Retrieval Paradigms?

To determine whether the effectiveness rankings observed in in-corpus retrieval generalize to in-document retrieval, we evaluated all six segmentation methods on the GutenQA dataset using the same four embedding models under both pre-embedding and contextualized chunking paradigms. This research question tests whether segmentation strategy preferences are fundamental or paradigm-dependent.

**Segmentation effectiveness rankings show dramatic inversion between retrieval paradigms.** Contrary to in-corpus retrieval results, methods that excel on BEIR fall short on GutenQA, while those underperforming on multi-document tasks rank highly on single-document retrieval. Proposition-based chunking delivers the highest GutenQA DCG@10 scores for three of the four embedding models (e.g., 0.5556 with Jina-v2 and 0.5530 with E5-base), yet it yields the lowest average nDCG@10 on BEIR in every case, trailing paragraph chunking by 18% for Jina-v2 (0.3487 vs. 0.4303) and 22% for E5-base (0.3883 vs. 0.4997). Similarly, LumberChunker leads the GutenQA rankings for Jina-v3 (0.5693 DCG@10) but drops to fourth place on BEIR with 0.4463 average nDCG@10. In sharp contrast, simple heuristics such as paragraph and fixed-size chunking rarely achieve top scores on GutenQA but consistently occupy the top two ranks across all BEIR datasets. This dramatic reversal indicates that retrieval paradigm fundamentally changes which segmentation strategies are effective.

**Paradigm-specific optimization creates poor transferability.** The inversion of effectiveness rankings demonstrates that chunking strategies optimized for precise passage localization within single documents do not generalize to open-domain multi-document retrieval. Proposition-based chunking's strength lies in creating highly specific, semantically-bounded chunks that facilitate exact fact localization. However, this same specificity becomes a liability in in-corpus retrieval, where the retriever must first identify relevant documents among thousands of candidates. Conversely, coarser structural boundaries preserve semantic context needed for broad relevance matching across a corpus, even though they sacrifice some in-document precision.

Table 1: Effectiveness comparison across different models, chunking strategies, and datasets using DCG@10 for GutenQA and nDCG@10 for other datasets (regular encoder)

| | | ***document*** | ***corpus*** | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | | GutenQA | fiqa | nfcorpus | scifact | trec-covid | arguana | scidocs | Avg |
| **Jina-v2** | Paragraph | 0.3893 | **0.3347** | **0.3042** | 0.6389 | 0.6518 | 0.4682 | 0.1843 | **0.4303** |
| | Sentence | 0.3261 | 0.3096 | 0.2847 | **0.6548** | **0.6674** | 0.4675 | 0.1732 | 0.4262 |
| | Fixed-Size | 0.3889 | 0.3339 | 0.2604 | 0.6350 | 0.6437 | **0.4706** | **0.1853** | 0.4215 |
| | Semantic | 0.3858 | 0.3030 | 0.2746 | 0.6445 | 0.6618 | 0.4022 | 0.1760 | 0.4104 |
| | Lumber | 0.4782 | 0.2754 | 0.2887 | 0.6540 | 0.6456 | 0.3906 | 0.1721 | 0.4044 |
| | Proposition | **0.5556** | 0.2186 | 0.2995 | 0.6331 | 0.4292 | 0.3681 | 0.1436 | 0.3487 |
| | **Avg** | 0.4207 | 0.2959 | 0.2854 | 0.6434 | 0.6166 | 0.4279 | 0.1724 | 0.4069 |
| **Jina-v3** | Paragraph | 0.4574 | **0.4747** | **0.3663** | 0.7247 | **0.7721** | **0.4327** | 0.1984 | **0.4948** |
| | Sentence | 0.3877 | 0.4333 | 0.3566 | 0.7163 | 0.7135 | 0.4256 | 0.1888 | 0.4723 |
| | Fixed-Size | 0.4827 | 0.4728 | 0.3596 | **0.7262** | 0.7557 | 0.4325 | **0.1985** | 0.4909 |
| | Semantic | 0.4105 | 0.4396 | 0.3596 | 0.7103 | 0.7474 | 0.3894 | 0.1890 | 0.4726 |
| | Lumber | **0.5693** | 0.4028 | 0.3534 | 0.6845 | 0.6900 | 0.3671 | 0.1797 | 0.4463 |
| | Proposition | 0.5484 | 0.2858 | 0.3579 | 0.6690 | 0.5756 | 0.3044 | 0.1569 | 0.3916 |
| | **Avg** | 0.4760 | 0.4182 | 0.3589 | 0.7052 | 0.7091 | 0.3920 | 0.1852 | 0.4614 |
| **Nomic** | Paragraph | 0.4893 | 0.3802 | 0.3509 | 0.7047 | 0.7652 | 0.4904 | 0.1808 | 0.4787 |
| | Sentence | 0.4146 | 0.3620 | 0.3502 | **0.7150** | 0.7472 | **0.5084** | 0.1732 | 0.4760 |
| | Fixed-Size | 0.5104 | **0.3839** | 0.3492 | 0.7050 | **0.7859** | 0.4915 | **0.1812** | **0.4828** |
| | Semantic | 0.4274 | 0.3562 | 0.3506 | 0.7040 | 0.7431 | 0.4434 | 0.1707 | 0.4613 |
| | Lumber | **0.6048** | 0.3347 | **0.3513** | 0.7093 | 0.7193 | 0.4315 | 0.1673 | 0.4522 |
| | Proposition | 0.5827 | 0.2575 | 0.3358 | 0.6853 | 0.5580 | 0.3693 | 0.1441 | 0.3916 |
| | **Avg** | 0.5049 | 0.3457 | 0.3480 | 0.7039 | 0.7198 | 0.4557 | 0.1696 | 0.4571 |
| **E5-Large** | Paragraph | 0.4663 | **0.4617** | 0.3566 | **0.7129** | 0.6634 | **0.5999** | 0.2038 | **0.4997** |
| | Sentence | 0.4063 | 0.4315 | 0.3583 | 0.6873 | 0.6712 | 0.5646 | 0.1836 | 0.4828 |
| | Fixed-Size | 0.4862 | 0.4599 | 0.3495 | 0.7078 | 0.6590 | 0.5987 | **0.2058** | 0.4968 |
| | Semantic | 0.3186 | 0.4173 | 0.3583 | 0.6892 | 0.6784 | 0.5540 | 0.1888 | 0.4810 |
| | Lumber | 0.5234 | 0.3739 | 0.3531 | 0.6748 | **0.6931** | 0.5233 | 0.1792 | 0.4663 |
| | Proposition | **0.5530** | 0.2522 | **0.3602** | 0.6851 | 0.5611 | 0.3163 | 0.1551 | 0.3883 |
| | **Avg** | 0.4590 | 0.3994 | 0.3560 | 0.6928 | 0.6544 | 0.5261 | 0.1861 | 0.4691 |

**No universal segmentation strategy satisfies both retrieval paradigms.** The lack of a method performing strongly across both settings definitively answers RQ2: segmentation strategy effectiveness is paradigm-dependent, not fundamental. This finding has critical implications for practitioners deploying retrieval systems: a single chunking configuration cannot effectively serve both in-corpus and in-document retrieval tasks. Systems requiring both capabilities must either maintain separate chunking pipelines or accept suboptimal performance on one retrieval paradigm.

Table 2: Effectiveness comparison across different models, chunking strategies, and datasets using DCG@10 for GutenQA and nDCG@10 for other datasets (late encoder)

| | | ***document*** | ***corpus*** | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | | GutenQA | fiqa | nfcorpus | scifact | trec-covid | arguana | scidocs | Avg |
| **Jina-v2** | Paragraph | 0.3476 | 0.3346 | 0.3041 | 0.6397 | 0.6618 | 0.4666 | 0.1854 | 0.4320 |
| | Sentence | 0.3084 | **0.3392** | 0.3033 | 0.6583 | **0.6791** | 0.4764 | 0.1839 | **0.4400** |
| | Fixed-Size | 0.3799 | 0.3351 | 0.3003 | 0.6448 | 0.6649 | 0.4685 | **0.1858** | 0.4332 |
| | Semantic | 0.4041 | 0.2195 | 0.2900 | 0.6489 | 0.6076 | 0.4715 | 0.1836 | 0.4035 |
| | Lumber | 0.4290 | 0.2223 | 0.3079 | 0.6527 | 0.6516 | 0.4759 | 0.1855 | 0.4160 |
| | Proposition | **0.5280** | 0.3042 | **0.3210** | **0.6632** | 0.4884 | **0.4869** | 0.1655 | 0.4049 |
| | **Avg** | 0.3995 | 0.2925 | 0.3044 | 0.6513 | 0.6256 | 0.4743 | 0.1816 | 0.4216 |
| **Jina-v3** | Paragraph | 0.2816 | 0.4306 | 0.3442 | 0.6826 | 0.6921 | **0.4732** | 0.2034 | 0.4710 |
| | Sentence | 0.2524 | **0.4355** | 0.3443 | 0.6841 | **0.7046** | 0.4605 | 0.2011 | **0.4717** |
| | Fixed-Size | 0.3411 | 0.4308 | 0.3442 | 0.6856 | 0.6881 | 0.4732 | **0.2036** | 0.4709 |
| | Semantic | 0.3356 | 0.2895 | 0.3474 | 0.6880 | 0.6527 | 0.4729 | 0.2019 | 0.4421 |
| | Lumber | 0.3789 | 0.2857 | 0.3506 | **0.6947** | 0.6826 | 0.4691 | 0.2020 | 0.4474 |
| | Proposition | **0.4984** | 0.3670 | **0.3551** | 0.6937 | 0.5229 | 0.4328 | 0.1883 | 0.4266 |
| | **Avg** | 0.3480 | 0.3732 | 0.3476 | 0.6881 | 0.6571 | 0.4636 | 0.2001 | 0.4550 |
| **Nomic** | Paragraph | 0.1839 | **0.3801** | 0.3508 | 0.7057 | 0.7361 | 0.4936 | **0.1808** | 0.4745 |
| | Sentence | 0.1557 | 0.3777 | 0.3536 | 0.7129 | **0.7517** | 0.4883 | 0.1778 | **0.4770** |
| | Fixed-Size | 0.2127 | 0.3796 | 0.3515 | 0.7058 | 0.7316 | **0.4941** | 0.1806 | 0.4738 |
| | Semantic | 0.2435 | 0.2804 | 0.3535 | 0.7036 | 0.7052 | 0.4890 | 0.1786 | 0.4517 |
| | Lumber | 0.2395 | 0.2757 | **0.3604** | **0.7191** | 0.6790 | 0.4823 | 0.1796 | 0.4494 |
| | Proposition | **0.4213** | 0.3145 | 0.3515 | 0.7124 | 0.4502 | 0.4773 | 0.1530 | 0.4098 |
| | **Avg** | 0.2428 | 0.3347 | 0.3535 | 0.7099 | 0.6756 | 0.4874 | 0.1751 | 0.4560 |
| **E5-Large** | Paragraph | 0.4223 | **0.4613** | **0.3576** | **0.7152** | **0.6630** | **0.5994** | **0.2064** | **0.5005** |
| | Sentence | 0.3621 | 0.4608 | 0.3553 | 0.7030 | 0.6479 | 0.5945 | 0.2061 | 0.4946 |
| | Fixed-Size | 0.4863 | 0.4601 | 0.3510 | 0.7095 | 0.6615 | 0.5978 | 0.2063 | 0.4977 |
| | Semantic | 0.3259 | 0.3227 | 0.3549 | 0.7006 | 0.6430 | 0.5923 | 0.2055 | 0.4698 |
| | Lumber | **0.5245** | 0.3015 | 0.3518 | 0.6976 | 0.6137 | 0.5922 | 0.2050 | 0.4603 |
| | Proposition | 0.5117 | 0.3306 | 0.3405 | 0.6427 | 0.4160 | 0.5056 | 0.1826 | 0.4030 |
| | **Avg** | 0.4388 | 0.3895 | 0.3518 | 0.6948 | 0.6075 | 0.5803 | 0.2020 | 0.4710 |

### 4.3 RQ3: Effectiveness of Contextualized Chunking

To assess whether contextualized chunking (late encoding) provides consistent improvements across segmentation methods and retrieval tasks, we compared late encoding to pre-embedding approaches for all six segmentation strategies across both in-corpus (BEIR) and in-document (GutenQA) retrieval settings. This research question evaluates whether contextualized chunking is a universal improvement or highly dependent on specific configurations.

**Contextualized chunking provides significant but highly method-dependent gains on in-corpus retrieval.** Proposition-based chunking experiences dramatic improvements across BEIR datasets when paired with late encoding, gaining +18.74% on average for Jina-v2, +14.06% for Jina-v3, +7.82% for Nomic, and +11.87% for E5-base. These gains are particularly pronounced on challenging domains (e.g., +39.17% on FiQA with Jina-v2, +42.20% on ArguAna with Jina-v3, +59.84% on ArguAna with E5-base). In contrast, simple heuristics like paragraph and fixed-size chunking remain largely stable on BEIR, with average changes within ±0.5% across most embedding models. This dichotomy reveals that contextualized embeddings selectively benefit semantically-structured chunks while providing minimal benefit to structurally-simple ones.

**Contextualized chunking severely degrades performance on in-document retrieval across all methods.** While LLM-guided methods gain substantially on in-corpus tasks, every segmentation strategy experiences severe performance loss on GutenQA when switched to late encoding. Paragraph chunking suffers the most extreme degradation (−62.41% DCG@10 with Nomic, −38.43% with Jina-v3, −10.72% with Jina-v2), indicating that full-document context actively harms passage-level precision. Even proposition-based chunking, which thrives on BEIR with contextualization, still loses performance on GutenQA (−27.70% with Nomic, −9.12% with Jina-v3). This universal degradation on in-document retrieval demonstrates a fundamental incompatibility between contextualized embeddings and fine-grained information localization tasks.

**Contextualized chunking effectiveness is highly contingent on both chunking strategy and dataset domain.** Beyond the in-corpus versus in-document distinction, contextualized chunking's impact varies dramatically by dataset type. Proposition-based chunking achieves its largest gains on argumentative corpora (ArguAna: +32.26% for Jina-v2, +42.20% for Jina-v3, +29.26% for Nomic, +59.84% for E5-base) but suffers regressions on structured scientific datasets (TREC-COVID: −9.16% for Jina-v3, −25.85% for E5-base). LumberChunker similarly exhibits domain-specific swings, gaining +27.78% on ArguAna with Jina-v3 yet losing −29.08% on FiQA. These patterns suggest that contextualized embeddings are most effective when documents contain complex argumentative structures requiring global context, but provide diminishing returns or cause harm on well-structured technical content.

**Contextualized chunking is not universally beneficial; its value depends on matching method to task.** Overall, contextualized chunking delivers modest average improvements across BEIR (ranging from +0.84% to +4.75% depending on embedding model) but at severe cost to in-document retrieval per-

formance (averaging −52.39% loss for Nomic, −27.25% for Jina-v3, −4.83% for Jina-v2, −4.22% for E5-base). These results directly answer RQ3: contextualized chunking is not a universal upgrade but rather a method-specific tool. LLM-guided strategies like proposition-based chunking unlock their full potential when paired with contextualized embeddings on in-corpus tasks, while structural heuristics derive minimal benefit. Critically, any system supporting both in-corpus and in-document retrieval cannot use contextualized chunking without accepting substantial performance degradation on one paradigm. Practitioners must make an explicit trade-off: preserve in-document retrieval performance with pre-embedding approaches, or sacrifice document-level precision to maximize corpus-level retrieval gains for LLM-guided methods.

### 4.4 RQ4: Other Factors

## 5 Conclusions

## Acknowledgment

## References

Table 3: Late-encoder effectiveness relative to the regular encoder (baseline). Values show percentage change (light green=improvement, light red=degradation)

| | | ***document*** | ***corpus*** | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | | GutenQA | fiqa | nfcorpus | scifact | trec-covid | arguana | scidocs | Avg |
| **Jina-v2** | Paragraph | -10.72 | -0.02 | -0.03 | +0.12 | +1.53 | -0.33 | +0.61 | +0.31 |
| | Sentence | -5.43 | +9.57 | +6.52 | +0.52 | +1.76 | +1.89 | +6.17 | +4.40 |
| | Fixed-Size | -2.32 | +0.35 | **+15.32** | +1.55 | +3.30 | -0.44 | +0.27 | +3.39 |
| | Semantic | **+4.75** | -27.57 | +5.59 | +0.69 | -8.19 | +17.22 | +4.31 | -1.32 |
| | Lumber | -10.30 | -19.29 | +6.66 | -0.19 | +0.93 | +21.85 | +7.80 | +2.96 |
| | Proposition | -4.98 | **+39.17** | +7.18 | **+4.75** | **+13.79** | **+32.26** | **+15.29** | **+18.74** |
| | **Avg** | -4.83 | +0.37 | +6.87 | +1.24 | +2.19 | +12.07 | +5.74 | +4.75 |
| **Jina-v3** | Paragraph | -38.43 | -9.28 | -6.02 | -5.81 | -10.37 | +9.37 | +2.52 | -3.27 |
| | Sentence | -34.89 | +0.51 | -3.43 | -4.49 | -1.25 | +8.20 | +6.55 | +1.01 |
| | Fixed-Size | -29.35 | -8.88 | -4.27 | -5.58 | -8.95 | +9.40 | +2.57 | -2.62 |
| | Semantic | -18.24 | -34.15 | -3.41 | -3.14 | -12.67 | +21.43 | +6.85 | -4.18 |
| | Lumber | -33.46 | -29.08 | -0.78 | +1.48 | -1.08 | +27.78 | +12.40 | +1.79 |
| | Proposition | -9.12 | **+28.41** | -0.78 | **+3.70** | -9.16 | **+42.20** | **+20.01** | **+14.06** |
| | **Avg** | -27.25 | -8.75 | -3.11 | -2.31 | -7.25 | +19.73 | +8.48 | +1.13 |
| **Nomic** | Paragraph | -62.41 | -0.03 | -0.03 | +0.14 | -3.80 | +0.65 | +0.02 | -0.51 |
| | Sentence | -62.46 | +4.33 | +0.98 | -0.28 | **+0.60** | -3.96 | +2.66 | +0.72 |
| | Fixed-Size | -58.33 | -1.10 | +0.63 | +0.11 | -6.91 | +0.51 | -0.36 | -1.19 |
| | Semantic | -43.04 | -21.28 | +0.82 | -0.04 | -5.09 | +10.29 | +4.62 | -1.78 |
| | Lumber | -60.40 | -17.64 | +2.61 | +1.39 | -5.59 | +11.78 | **+7.35** | -0.02 |
| | Proposition | -27.70 | **+22.15** | **+4.68** | **+3.95** | -19.32 | **+29.26** | +6.22 | **+7.82** |
| | **Avg** | -52.39 | -2.26 | +1.62 | +0.88 | -6.69 | +8.09 | +3.42 | +0.84 |
| **E5-Large** | Paragraph | -9.43 | -0.09 | +0.27 | +0.32 | -0.05 | -0.08 | +1.27 | +0.27 |
| | Sentence | -10.89 | +6.80 | -0.84 | +2.29 | -3.48 | +5.29 | +12.28 | +3.72 |
| | Fixed-Size | +0.01 | +0.05 | **+0.42** | +0.25 | **+0.38** | -0.14 | +0.25 | +0.20 |
| | Semantic | **+2.29** | -22.66 | -0.96 | +1.66 | -5.22 | +6.92 | +8.83 | -1.90 |
| | Lumber | +0.20 | -19.37 | -0.39 | **+3.37** | -11.46 | +13.18 | +14.38 | -0.05 |
| | Proposition | -7.47 | **+31.09** | -5.47 | -6.18 | -25.85 | **+59.84** | **+17.77** | **+11.87** |
| | **Avg** | -4.22 | -0.69 | -1.16 | +0.28 | -7.61 | +14.17 | +9.13 | +2.35 |